



TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DEL VALLE DE OAXACA

Administración y Organización de Datos

PROFESOR:

BENEDICTO RAMIREZ SANTIAGO

ALUMNO:

JESÚS UBLADO PEDRO ALAVEZ

INGENIERÍA INFORMÁTICA

GRUPO: 4° A

CUARTO SEMESTRE

UNIDAD 2:

ACTIVIDAD: **Actividad Sistema de Inventario**

NAZARENO, XOXOCOTLÁN, OAXACA, MÉXICO A 29 DE MAYO 2026

INDICE

INTRODUCCIÓN	3
DESARROLLO	4
Modelo	4
Clase Producto	4
Clase Archivoinventario	6
ModeloGuardarProducto	7
ModeloMostrarInventario.....	8
ModeloBuscarProducto	9
ModeloActualizarProducto	10
ModeloEliminarProducto	11
ModeloReporteProducto	12
Controlador.....	13
Vista.....	15
CONCLUSIÓN	23

INTRODUCCIÓN

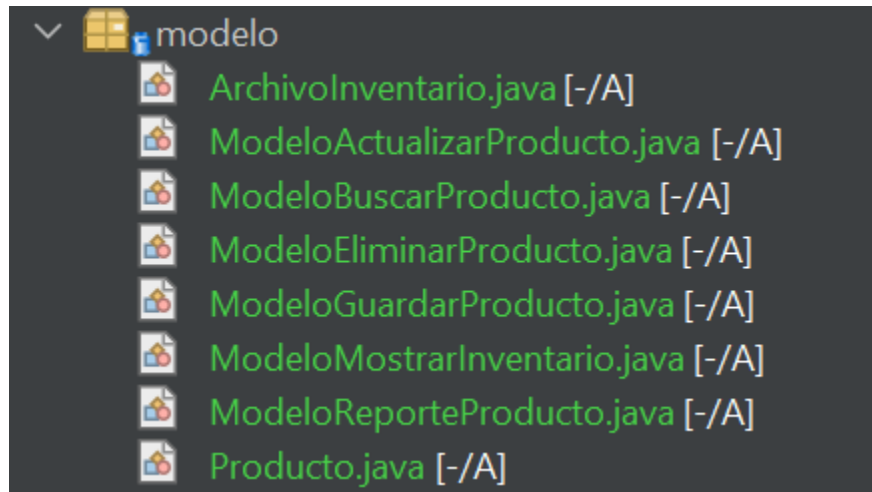
El presente proyecto implementa un Sistema de Inventario en Java utilizando la arquitectura MVC (Modelo-Vista-Controlador). El sistema permite gestionar productos mediante operaciones CRUD completas: guardar, mostrar, buscar, actualizar y eliminar productos, además de generar reportes en archivo de texto.

Los datos se persisten en un archivo binario de acceso directo (inventario.dat) utilizando la clase `RandomAccessFile`, lo que permite leer y escribir registros de tamaño fijo en posiciones específicas del archivo.

DESARROLLO

Modelo

El paquete modelo contiene todas las clases encargadas de la lógica de negocio y el acceso al archivo de datos.



Clase Producto

Define la estructura de un producto con sus atributos y constantes de tamaño fijo para la escritura en el archivo binario.

```
package modelo;

public class Producto {
    public int id;
    public String nombre;
    public String descripcion;
    public int existencia;
    public boolean estado;

    public static final int TAM_NOMBRE = 20;
    public static final int TAM_DESC = 30;
    public static final int TAM_REG = 4 + (2 * TAM_NOMBRE) + (2 * TAM_DESC) + 4 + 1;

    public Producto(int id, String nombre, String descripcion,
                    int existencia, boolean estado) {
        this.id = id;
        this.nombre = ajustar(nombre, TAM_NOMBRE);
        this.descripcion = ajustar(descripcion, TAM_DESC);
        this.existencia = existencia;
    }
}
```

```
        this.estado = estado;
    }

    public static String ajustar(String texto, int longitud) {
        if (texto.length() > longitud) {
            return texto.substring(0, longitud);
        }
        while (texto.length() < longitud) {
            texto += " ";
        }
        return texto;
    }
}
```

Clase ArchivoInventario

Gestiona el acceso de bajo nivel al archivo binario. Implementa los métodos de lectura y escritura de registros usando `RandomAccessFile`.

```
package modelo;

import java.io.IOException;
import java.io.RandomAccessFile;

public class ArchivoInventario {
    RandomAccessFile archivo;

    public ArchivoInventario(String nombre) throws IOException {
        archivo = new RandomAccessFile(nombre, "rw");
    }

    public void guardar(Producto p, int pos) throws IOException {
        archivo.seek(pos * Producto.TAM_REG);
        archivo.writeInt(p.id);
        archivo.writeChars(p.nombre);
        archivo.writeChars(p.descripcion);
        archivo.writeInt(p.existencia);
        archivo.writeBoolean(p.estado);
    }

    public Producto leer(int pos) throws IOException {
        if ((pos + 1) * Producto.TAM_REG > archivo.length()) return null;
        archivo.seek(pos * Producto.TAM_REG);
        int id = archivo.readInt();
        char[] nom = new char[Producto.TAM_NOMBRE];
        for (int i = 0; i < nom.length; i++) nom[i] = archivo.readChar();
        char[] desc = new char[Producto.TAM_DESC];
        for (int i = 0; i < desc.length; i++) desc[i] = archivo.readChar();
        int existencia = archivo.readInt();
        boolean estado = archivo.readBoolean();
        return new Producto(id, new String(nom).trim(),
                           new String(desc).trim(), existencia, estado);
    }

    public int totalRegistros() throws IOException {
        return (int) (archivo.length() / Producto.TAM_REG);
    }
}
```

ModeloGuardarProducto

Contiene la lógica para guardar un nuevo producto en el archivo, validando que el ID no exista previamente entre los registros activos.

```
package modelo;

import java.io.IOException;

public class ModeloGuardarProducto {
    ArchivoInventario archivo;

    public ModeloGuardarProducto() throws IOException {
        archivo = new ArchivoInventario("inventario.dat");
    }

    public boolean guardarProducto(int id, String nombre,
                                    String descripcion, int existencia) {
        try {
            int total = archivo.totalRegistros();
            for (int i = 0; i < total; i++) {
                Producto p = archivo.leer(i);
                if (p != null && p.estado && p.id == id) return false;
            }
            Producto nuevo = new Producto(id, nombre, descripcion,
                                           existencia, true);
            archivo.guardar(nuevo, total);
            return true;
        } catch (IOException ex) {
            return false;
        }
    }
}
```


ModeloMostrarInventario

Lee todos los registros activos del archivo y los retorna como matriz de objetos para mostrarlos en la tabla de la interfaz gráfica.

```
package modelo;

import java.io.IOException;
import java.util.ArrayList;

public class ModeloMostrarInventario {
    ArchivoInventario archivo;
    private ArrayList<Integer> posiciones;

    public ModeloMostrarInventario() throws IOException {
        archivo = new ArchivoInventario("inventario.dat");
        posiciones = new ArrayList<>();
    }

    public Object[][] mostrarInventario() {
        try {
            posiciones.clear();
            int total = archivo.totalRegistros();
            ArrayList<Object[]> filas = new ArrayList<>();
            for (int i = 0; i < total; i++) {
                Producto p = archivo.leer(i);
                if (p != null && p.estado) {
                    filas.add(new Object[]{ p.id, p.nombre,
                        p.descripcion, p.existencia, p.estado });
                    posiciones.add(i);
                }
            }
            Object[][] datos = new Object[filas.size()][5];
            for (int i = 0; i < filas.size(); i++) datos[i] = filas.get(i);
            return datos;
        } catch (IOException ex) {
            return new Object[0][0];
        }
    }

    public ArrayList<Integer> getPosiciones() { return posiciones; }
}
```


ModeloBuscarProducto

Permite buscar un producto por su ID, retornando el objeto Producto encontrado y su posición física en el archivo.

```
Package modelo;

import java.io.IOException;

public class ModeloBuscarProducto {
    ArchivoInventario archivo;

    public ModeloBuscarProducto() throws IOException {
        archivo = new ArchivoInventario("inventario.dat");
    }

    public Producto buscarProducto(int idBuscar) {
        try {
            int total = archivo.totalRegistros();
            for (int i = 0; i < total; i++) {
                Producto p = archivo.leer(i);
                if (p != null && p.estado && p.id == idBuscar) return p;
            }
        } catch (IOException ex) { return null; }
        return null;
    }

    public int buscarPosicionProducto(int idBuscar) {
        try {
            int total = archivo.totalRegistros();
            for (int i = 0; i < total; i++) {
                Producto p = archivo.leer(i);
                if (p != null && p.estado && p.id == idBuscar) return i;
            }
        } catch (IOException ex) { return -1; }
        return -1;
    }
}
```

ModeloActualizarProducto

Sobreescribe un registro existente en la posición indicada del archivo con los nuevos datos proporcionados.

```
package modelo;

import java.io.IOException;

public class ModeloActualizarProducto {
    ArchivoInventario archivo;

    public ModeloActualizarProducto() throws IOException {
        archivo = new ArchivoInventario("inventario.dat");
    }

    public boolean actualizarProducto(int posicion, int id,
        String nombre, String descripcion, int existencia) {
        try {
            Producto p = new Producto(id, nombre, descripcion,
                existencia, true);
            archivo.guardar(p, posicion);
            return true;
        } catch (IOException ex) {
            return false;
        }
    }
}
```

ModeloEliminarProducto

Realiza una eliminación lógica del producto cambiando su campo estado a false, sin eliminar físicamente el registro del archivo.

```
package modelo;

import java.io.IOException;

public class ModeloEliminarProducto {
    ArchivoInventario archivo;

    public ModeloEliminarProducto() throws IOException {
        archivo = new ArchivoInventario("inventario.dat");
    }

    public boolean eliminarProducto(int posicion) {
        try {
            Producto p = archivo.leer(posicion);
            if (p == null) return false;
            p.estado = false;
            archivo.guardar(p, posicion);
            return true;
        } catch (IOException ex) {
            return false;
        }
    }
}
```

ModeloReporteProducto

Genera un reporte en formato de texto plano (reporte_inventario.txt) con todos los productos activos del inventario.

```
package modelo;

import java.io.IOException;
import java.io.PrintWriter;

public class ModeloReporteProducto {
    ArchivoInventario archivo;

    public ModeloReporteProducto() throws IOException {
        archivo = new ArchivoInventario("inventario.dat");
    }

    public boolean generarReporte() {
        try {
            PrintWriter pw = new PrintWriter("reporte_inventario.txt");
            pw.println("REPORTE DE INVENTARIO");
            pw.println("-----");
            pw.println();
            int total = archivo.totalRegistros();
            for (int i = 0; i < total; i++) {
                Producto p = archivo.leer(i);
                if (p != null && p.estado) {
                    pw.println("ID: " + p.id);
                    pw.println("Nombre: " + p.nombre);
                    pw.println("Descripcion: " + p.descripcion);
                    pw.println("Existencia: " + p.existencia);
                    pw.println("Estado: " + p.estado);
                    pw.println("-----");
                }
            }
            pw.close();
            return true;
        } catch (IOException ex) {
            return false;
        }
    }
}
```

Controlador

El paquete controlador contiene la clase ControladorInventario, que actúa como intermediario entre la vista y los modelos, manejando la lógica de cada acción del usuario.

```
package controlador;

import java.io.IOException;
import java.util.ArrayList;
import javax.swing.JOptionPane;
import javax.swing.table.DefaultTableModel;
import modelo.*;
import vista.UIPrincipal;

public class ControladorInventario {
    UIPrincipal vista;
    ModeloGuardarProducto objModGuardar;
    ModeloMostrarInventario objModMostrar;
    ModeloBuscarProducto objModBuscar;
    ModeloActualizarProducto objModActualizar;
    ModeloEliminarProducto objModEliminar;
    ModeloReporteProducto objModReporte;
    ArrayList<Integer> posiciones;

    public ControladorInventario(UIPrincipal vista) throws IOException {
        this.vista = vista;
        this.objModGuardar = new ModeloGuardarProducto();
        this.objModMostrar = new ModeloMostrarInventario();
        this.objModBuscar = new ModeloBuscarProducto();
        this.objModActualizar = new ModeloActualizarProducto();
        this.objModEliminar = new ModeloEliminarProducto();
        this.objModReporte = new ModeloReporteProducto();
        this.posiciones = new ArrayList<>();
    }

    public void listarProductos() {
        String[] columnas = {"ID", "Nombre", "Descripcion", "Existencia", "Estado"};
        Object[][] datos = objModMostrar.mostrarInventario();
        DefaultTableModel modelo = new DefaultTableModel(datos, columnas) {
            public boolean isCellEditable(int f, int c) { return false; }
        };
        vista.getTblProductos().setModel(modelo);
        posiciones = objModMostrar.getPosiciones();
    }

    public void guardarProducto() {
        try {
            if (camposVacios()) {
                JOptionPane.showMessageDialog(vista, "Completa todos los campos");
                return;
            }
            int id = Integer.parseInt(vista.getTxtId().getText());
            String nombre = vista.getTxtNombre().getText();
            String descripcion = vista.getTxtDescripcion().getText();
            int existencia = Integer.parseInt(vista.getTxtExistencia().getText());
            boolean guardado = objModGuardar.guardarProducto(
                id, nombre, descripcion, existencia);
        }
    }
}
```

```

        if (guardado) {
            listarProductos(); vista.limpiarCajas();
            JOptionPane.showMessageDialog(vista, "Producto guardado");
        } else {
            JOptionPane.showMessageDialog(vista, "ID ya existe");
        }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(vista, "ID y existencia deben ser numeros");
    }
}

public void seleccionarProducto() {
    try {
        int fila = vista.getTblProductos().getSelectedRow();
        if (fila == -1) return;
        int id = Integer.parseInt(
            vista.getTblProductos().getValueAt(fila, 0).toString());
        Producto p = objModBuscar.buscarProducto(id);
        if (p != null) {
            vista.getTxtId().setText(String.valueOf(p.id));
            vista.getTxtNombre().setText(p.nombre);
            vista.getTxtDescripcion().setText(p.descripcion);
            vista.getTxtExistencia().setText(String.valueOf(p.existencia));
        }
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(vista, "Error al seleccionar");
    }
}

public void actualizarProducto() { /* ... implementacion completa ... */ }
public void eliminarProducto()    { /* ... implementacion completa ... */ }
public void buscarProducto()      { /* ... implementacion completa ... */ }
public void generarReporte() {
    boolean ok = objModReporte.generarReporte();
    JOptionPane.showMessageDialog(vista,
        ok ? "Reporte generado" : "Error al generar reporte");
}

private boolean camposVacios() {
    return vista.getTxtId().getText().trim().isEmpty()
        || vista.getTxtNombre().getText().trim().isEmpty()
        || vista.getTxtDescripcion().getText().trim().isEmpty()
        || vista.getTxtExistencia().getText().trim().isEmpty();
}
}

```


Vista

El paquete vista contiene la clase `UIPrincipal` que extiende `JFrame` y construye la interfaz gráfica del sistema con campos de texto, botones y la tabla de inventario.

```
package vista;

import controlador.ControladorInventario;
import java.io.IOException;
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class UIPrincipal extends javax.swing.JFrame {
    ControladorInventario controlador;
    private JTextField txtId, txtNombre, txtDescripcion, txtExistencia;
    private JTable tblProductos;
    private JButton btnGuardar, btnActualizar, btnEliminar,
        btnBuscar, btnReporte;

    public UIPrincipal() throws IOException {
        initComponents();
        controlador = new ControladorInventario(this);
        controlador.listarProductos();
        setLocationRelativeTo(null);
    }

    private void initComponents() {
        setTitle("Sistema de Inventario");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(750, 550);
        setLayout(new BorderLayout(10, 10));
        // ... construccion de la UI ...
        btnGuardar.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                controlador.guardarProducto();
            }
        });
        // ... demas listeners ...
    }

    public void limpiarCajas() {
        txtId.setText(""); txtNombre.setText("");
        txtDescripcion.setText(""); txtExistencia.setText("");
        tblProductos.clearSelection();
    }

    public JTable getTblProductos() { return tblProductos; }
    public JTextField getTxtId() { return txtId; }
    public JTextField getTxtNombre() { return txtNombre; }
    public JTextField getTxtDescripcion() { return txtDescripcion; }
    public JTextField getTxtExistencia() { return txtExistencia; }

    public static void main(String[] args) {
        java.awt.EventQueue.invokeLater(() -> {
            try { new UIPrincipal().setVisible(true); }
        });
    }
}
```



```
        catch (IOException ex) { ex.printStackTrace(); }  
    });  
}  
}
```

EVIDENCIAS

Sistema de Inventario

Datos del Producto

ID:

Nombre:

Descripción:

Existencia:

Guardar

Actualizar

Eliminar

Buscar

Reporte

Inventario

ID	Nombre	Descripción	Existencia	Estado
1212	loteria	JugueteMexicano	112	true

(Ejecución del Programa)

Sistema de Inventario

Datos del Producto

Inventario

ID: 1212

ID: 1111

Nombre: Valero Mexicano

Descripción: Juguete Mexicano

Existencia: 12

Guardar

Parte del Registro de un producto y notificación de validación cumplida es decir que si se realizo el registro

1111 Valero Mexicano Juguete Mexicano 12 true

ID:

Nombre:

Descripción:

Existencia:

Mensaje

Producto guardado correctamente

Aceptar

Modificación de Producto en este caso se agrego mas Stock

ID	Nombre	Descripción	Existencia	Estado
1212	loteria	Juguete Mexicano	112	true
1111	Valero Mexicano	Juguete Mexicano	30	true

ID:

Nombre:

Descripción:

Existencia:

Mensaje de proceso completado

ID	Nombre	Descripción	Existencia	Estado
1111	Valero Mexicano	Juguete Mexicano	20	true

Mensaje

 Producto actualizado correctamente

Proceso de eliminación de un artículo

Sistema de inventario

Datos del Producto

ID: 1212

Nombre: loteria

Descripción: Juguete Mexicano

Existencia: 112

Quitar

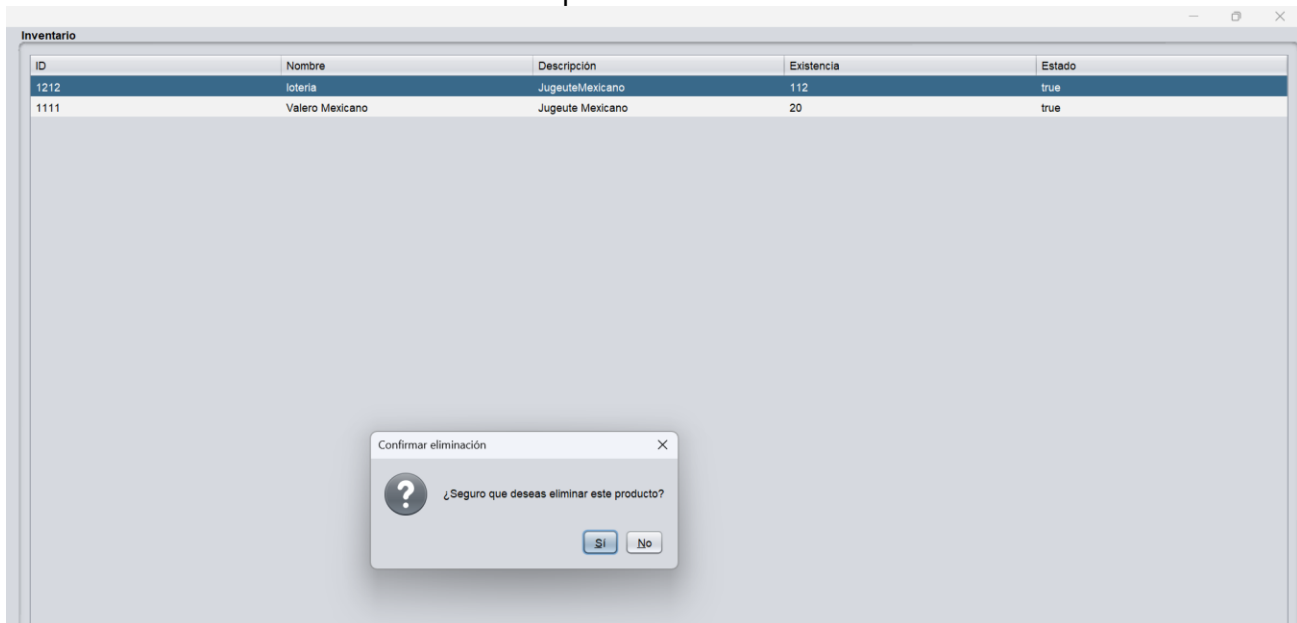
Actualizar

Eliminar

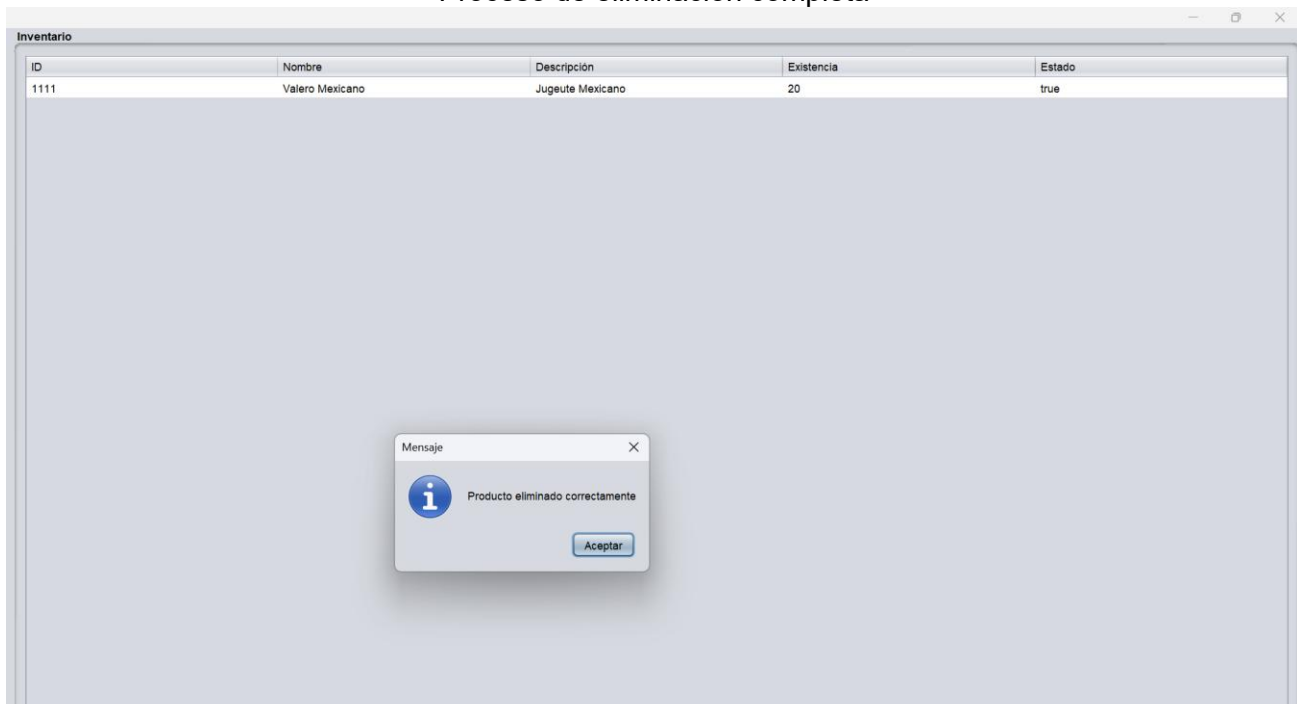
Inventario

ID	Nombre	Descripción	Existencia	Estado
1212	loteria	Juguete Mexicano	112	true
1111	Valero Mexicano	Juguete Mexicano	20	true

Eliminación con éxito pero con advertencia de eliminación



Proceso de eliminación completa



Proceso de búsqueda

ID:

Nombre:

Descripción:

Existencia:

Resultado de la búsqueda

Sistema de Inventario

Datos del Producto

Inventario

ID	Nombre	Descripción	Existencia	Estado
1111	Valero Mexicano	Juguete Mexicano	20	true

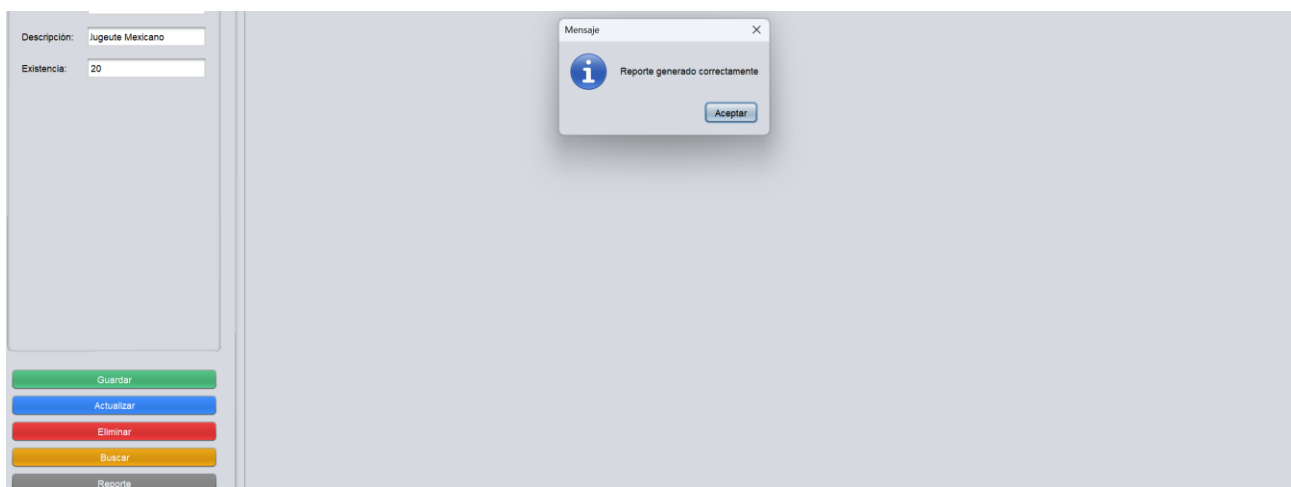
ID:

Nombre:

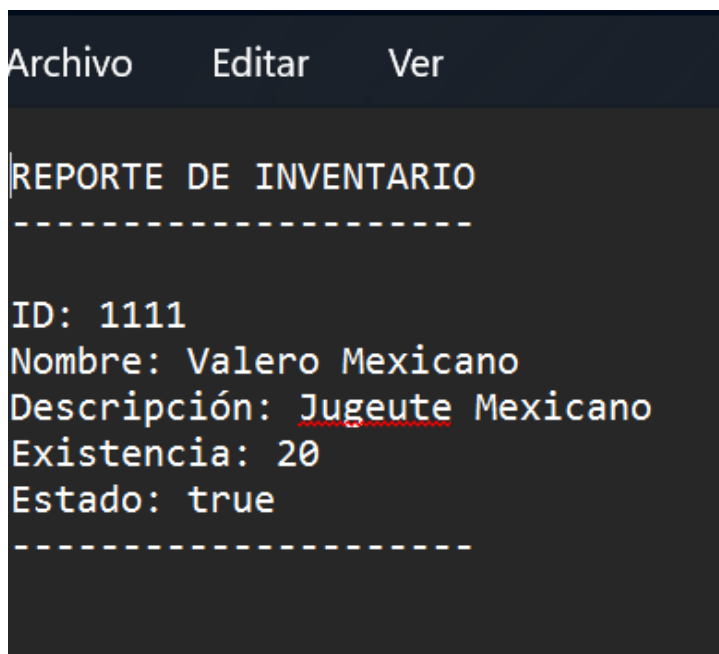
Descripción:

Existencia:

Proceso de Reporte generado



Carpeta del reporte generado





CONCLUSIÓN

El proyecto implementa correctamente los principios de la arquitectura MVC separando responsabilidades en tres capas bien definidas. El uso de archivos de acceso directo con `RandomAccessFile` permite operaciones eficientes sobre registros de tamaño fijo, haciendo posible acceder a cualquier posición del archivo en tiempo constante.

La eliminación lógica (`estado = false`) conserva la integridad del archivo sin reorganizarlo, lo que simplifica las operaciones de escritura. La interfaz gráfica en Swing ofrece una experiencia de usuario completa para todas las operaciones CRUD del inventario.

